

Learning Objectives

Students will be able to (1) compute entropy and information gain for decision tree splitting criteria, (2) evaluate the performance of various Decision Tree classifiers using `scikit-learn`, and (3) experiment with variations of the classifier.

1 Entropy and Information Gain (30 points)

Consider a small subset of the Titanic dataset with the following 8 passengers:

Passenger	Class (C)	Sex (S)	Age (G)	Survived (Y)
1	1st	Male	Adult	Yes
2	1st	Female	Adult	Yes
3	3rd	Male	Adult	No
4	3rd	Male	Child	No
5	2nd	Female	Adult	Yes
6	3rd	Female	Adult	No
7	1st	Female	Child	Yes
8	2nd	Male	Adult	No

Formulas

Entropy:

$$H(Y) = - \sum_{y \in Y} P(Y = y) \log_2(P(Y = y)) \quad (1)$$

Conditional entropy:

$$H(Y|A) = \sum_{\alpha \in A} P(A = \alpha) \cdot H(Y|A = \alpha) \quad (2)$$

$$H(Y|A = \alpha) = - \sum_{y \in Y} P(Y = y|A = \alpha) \log_2(P(Y = y|A = \alpha)) \quad (3)$$

Information Gain:

$$I(Y, A) = H(Y) - H(Y|A) \quad (4)$$

Where:

- $H(Y)$ is the entropy of the target Y
- $H(Y|A)$ is the conditional entropy after splitting on feature A

(a) **Initial entropy** (5 points):

Calculate the entropy $H(Y)$ of the target variable (Survived) for the entire dataset.

$$\begin{aligned}
 H(Y) &= - [P_{yes} \cdot \log_2(P_{yes}) + P_{no} \cdot \log_2(P_{no})] \\
 &= - \left[\frac{4}{8} \log_2 \frac{4}{8} + \frac{4}{8} \log_2 \frac{4}{8} \right] \\
 &= - \left[\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right]
 \end{aligned}$$

$$H(Y) = 1$$

(b) Information gain for Class (C) (10 points):

i. Calculate the entropy $H(Y|C)$ after splitting on the Class attribute.

$$\begin{aligned}
 C=0 &: 1st\ class: \\
 H(Y|C=0) &= -[P(Y=yes|C=0) \cdot \log_2 P(Y=yes|C=0) + \\
 &\quad P(Y=no|C=0) \cdot \log_2 P(Y=no|C=0)] \\
 &= -\left[\frac{3}{3} \log_2 \frac{3}{3} + \frac{0}{3} \log_2 \frac{0}{3}\right] \\
 &= 0 \\
 \\
 C=1 &: 2nd\ class: \\
 H(Y|C=1) &= -[P(Y=yes|C=1) \cdot \log_2 P(Y=yes|C=1) + \\
 &\quad P(Y=no|C=1) \cdot \log_2 P(Y=no|C=1)] \\
 &= -\left[\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}\right] \\
 &= 1 \\
 \\
 C=2 &: 3rd\ class: \\
 H(Y|C=2) &= -[P(Y=yes|C=2) \cdot \log_2 P(Y=yes|C=2) + \\
 &\quad P(Y=no|C=2) \cdot \log_2 P(Y=no|C=2)] \\
 &= -\left[\frac{0}{3} \log_2 \frac{0}{3} + \frac{3}{3} \log_2 \frac{3}{3}\right] \\
 &= 0 \\
 \\
 H(Y|C) &= P(C=0) H(Y|C=0) + P(C=1) H(Y|C=1) + \\
 &\quad P(C=2) H(Y|C=2) \\
 H(Y|C) &= (0) \frac{3}{8} + (1) \frac{2}{8} + (0) \frac{3}{8} \\
 \boxed{H(Y|C) = 0.25}
 \end{aligned}$$

ii. Calculate the information gain $I(Y, C)$ for the Class attribute.

$$\begin{aligned}
 I(Y, C) &= H(Y) - H(Y|C) \\
 I(Y, C) &= 1 - 0.25 \\
 \boxed{I(Y, C) = 0.75}
 \end{aligned}$$

(c) **Information gain for Age (G)** (10 points):

i. Calculate the entropy $H(Y|G)$ after splitting on the Age attribute.

$$\begin{aligned}
 G = 0 : \text{Adult} : \\
 H(Y|G=0) &= -[P(Y=yes|G=0) \cdot \log_2 P(Y=yes|G=0) + \\
 &\quad P(Y=no|G=0) \cdot \log_2 P(Y=no|G=0)] \\
 &= -\left[\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6}\right] \\
 &= 1 \\
 \\
 G = 1 : \text{Child} : \\
 H(Y|G=1) &= -[P(Y=yes|G=1) \cdot \log_2 P(Y=yes|G=1) + \\
 &\quad P(Y=no|G=1) \cdot \log_2 P(Y=no|G=1)] \\
 &= -\left[\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}\right] \\
 &= 1 \\
 \\
 H(Y|G) &= P(G=0) H(Y|G=0) + P(G=1) H(Y|G=1) \\
 H(Y|G) &= \frac{6}{8} (1) + \frac{2}{8} (1) = \boxed{1}
 \end{aligned}$$

ii. Calculate the information gain $I(Y, G)$ for the Age attribute.

$$\begin{aligned}
 I(Y, G) &= H(Y) - H(Y|G) \\
 I(Y, G) &= 1 - 1 \\
 I(Y, G) &= \boxed{0}
 \end{aligned}$$

(d) **Best split** (5 points): Given that the information gain for the Sex attribute $I(Y, S) = 0.189$, which attribute (Class, Sex, or Age) provides the highest information gain? Justify your answer.

Class provides the best split because it has the highest information gain, 0.75, compared with 0.189 for Sex and 0 for Age. This means Class reduces the uncertainty in the Survived target the most.

Programming

2 GenAI (5 points)

Read through the rest of the assignment.

- Write down at least one use case for LLM-based AI assistant (e.g., ChatGPT, Gemini, Claude, etc). Example questions include explaining a function from a library that you're not familiar with, providing sample code for how to use the function, or debugging your code.
- Try the question with at least two models from <https://arena.ai/text/side-by-side>. Reflect on the output. Which tools/models did you use? Which model gave you the preferred answer and why? Were they accurate? Helpful? Justify your answer.

I used an LLM to better understand how `DecisionTreeClassifier(criterion='entropy')` works in scikit-learn and how changing `max_depth` can affect overfitting. I compared GPT-5.2 High and Grok 4.20 Beta Reasoning. Both models accurately explained that entropy measures how mixed the classes are and that information gain is used to choose useful splits. They also both explained that increasing `max_depth` makes the tree more complex and can cause overfitting. I preferred GPT-5.2 High because its explanation was more precise, directly explained how scikit-learn uses binary splits and weighted child entropy, and it finished its response about five seconds faster. Grok was also helpful because its bias-variance table made the effect of tree depth easy to understand. Overall, both responses were accurate and helpful, but GPT-5.2 High was more efficient and easier for me to follow.

In homework 1, we used some baseline classifiers to predict passenger survivability on the Titanic. This assignment, we will use `scikit-learn` to train and evaluate a `DecisionTreeClassifier` on the data.¹

- PostCommit starter code: <https://lmu.postcommit.ai/c/cmsi-5350-fall-2026/assignments/79>
- code and data
 - code: `titanic.py`
 - data: `titanic_train.csv`, `titanic_test.csv`
- documentation
 - Decision Tree Classifier:
<http://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
 - Cross-Validation:
http://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html
 - Metrics:
http://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

Note that any portions of the code that you must modify have been indicated with `TODO`. Do not change any code outside of these blocks.

¹This assignment is adapted from course material by Jessica Wu at Harvey Mudd College, and from the Kaggle Titanic competition, available at <https://www.kaggle.com/c/titanic>. Some parts of the problem are copied verbatim from Kaggle.

3 Titanic (65 points)

Using the predictive capabilities of the `scikit-learn` package is very simple. In fact, it can be carried out in three simple steps: initializing the model, fitting it to the training data, and predicting new values. For this section, you will modify `titanic.py`.

- (a) **Decision Tree** (10 points): Train and evaluate a `DecisionTreeClassifier` (using the class from `scikit-learn` and referring to the documentation as needed). Make sure you initialize your classifier with the appropriate parameters; in particular, use the `entropy` criterion discussed in class. What's the training error?

0.014

Optional: If you want to look at the decision tree, you can use `GraphViz`,² an open source graph visualization software, and the `pydot` library,³ a Python interface to Graphviz's dot language.

- (b) **Error** (10 points): So far, we have looked only at training error, but as we learned in class, training error is a poor metric for evaluating classifiers. Let us use cross-validation instead.

Implement the missing portions of `error(...)` according to the provided specifications. To do this, generate a random 80/20 split of the training data, train a model on the 80% fraction, store the training and testing errors on the 80% and 20% fractions respectively, and repeat this 100 times to get an average result. You may find it helpful to use `train_test_split(...)` from `scikit-learn`. To ensure that we always get the same splits across different runs (and thus can compare the classifier results), **set the `random_state` parameter to be the trial number** (e.g., 42⁴). More information about pseudo-random number generators in the `random` module⁵ in Python.

Next, use your `error(...)` function to evaluate the training error and (cross-validation) test error for each of your three models (i.e., majority vote, random, and decision tree). What are the average training and test error of each of your classifiers on the Titanic dataset?

Majority Vote:
train 0.404, test 0.407

Random Classifier:
train 0.489, test 0.487

DT Classifier:
train 0.012, test 0.242

I fixed the decision tree's random state (`random_state=0`) so the results would be reproducible across runs.

²<http://www.graphviz.org/>

³<https://pypi.python.org/pypi/pydot>

⁴<https://medium.com/geekculture/the-story-behind-random-seed-42-in-machine-learning-b838c4ac290a>

⁵<https://docs.python.org/3/library/random.html>

- (c) **Overfitting** (15 points): One problem with decision trees is they can *overfit* to training data, yielding complex classifiers that do not generalize well to new data.

One way to prevent decision trees from overfitting is to limit their depth. Repeat your cross-validation experiments but for increasing depth limits, specifically, $1, 2, \dots, 20$. Then plot the average training error and test error against the depth limit. (Also plot the average test error for your baseline classifiers, i.e., majority vote and random. As the baseline classifiers are independent of the depth limit, their plots should be flat lines.) Include this plot in your writeup, making sure to label all axes and include a legend for your classifiers. What is the best depth limit to use for this data? Do you see overfitting? Justify your answers using the plot. *See the plot below for what to expect.*

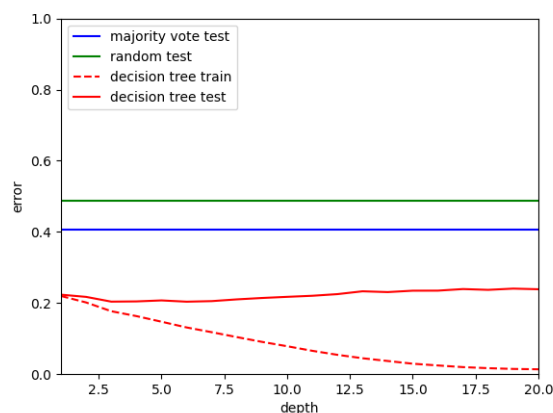
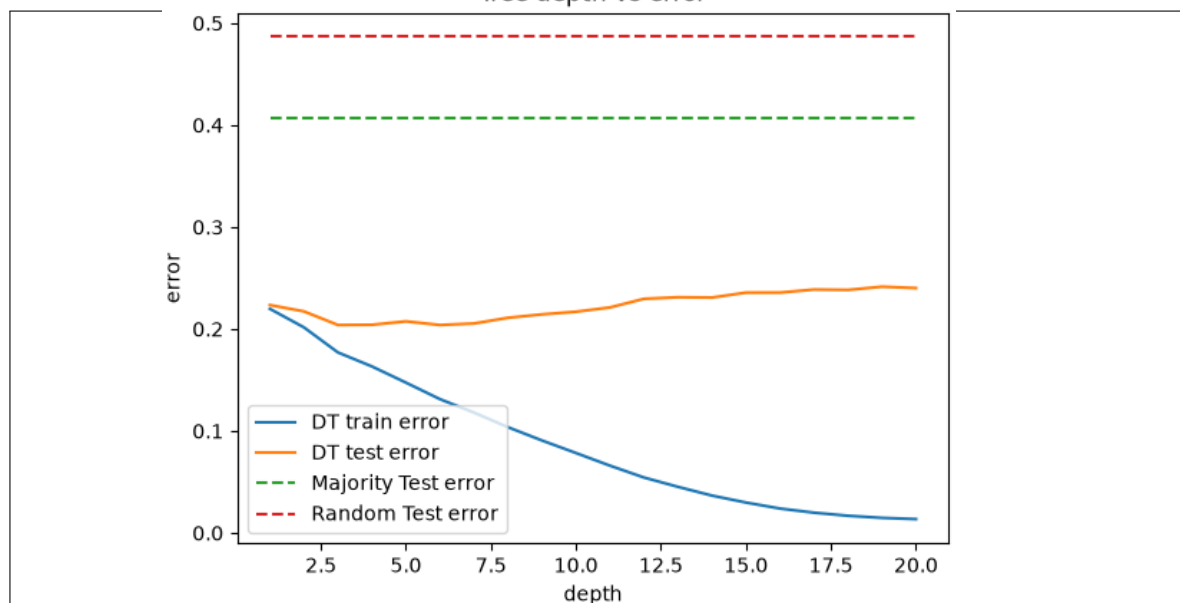


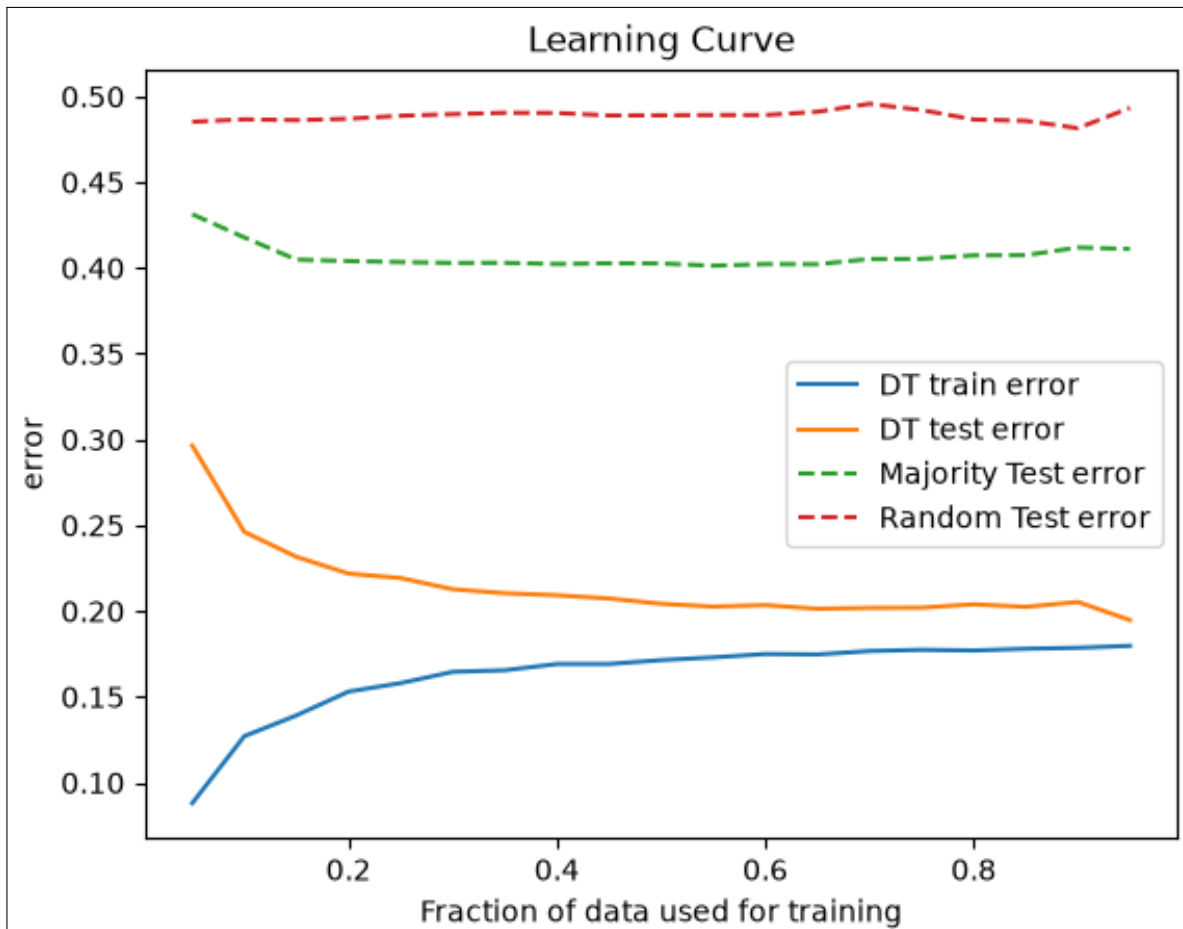
Figure 1: Tree depth vs error
Tree depth vs error



The best depth limit is 3, which gives the lowest average test error (0.204). Depth 6 gives the same test error, but I chose depth 3 because it achieves the same performance with a simpler tree. Yes, there is overfitting: as depth increases past about 3, training error keeps decreasing toward 0.014, while test error rises to about 0.240. The widening gap shows that deeper trees memorize the training data instead of learning patterns that generalize.

- (d) **Learning curve** (15 points): Another useful tool for evaluating classifiers is *learning curves*, which show how a classifier's performance (e.g. error) relates to experience (e.g. amount of training data).

Run another experiment using a decision tree with the best depth limit you found above. This time, vary the amount of training data by starting with splits of 0.05 (5% of the data used for training) and working up to splits of size 0.95 (95% of the data used for training) in increments of 0.05. Then plot the decision tree training and test error against the amount of training data. (Also plot the average test error for your two baseline classifiers.) Include this plot in your writeup, and provide a 1-2 sentence description of your observations.



As the training data increases, the decision tree's test error goes down (about 0.30 to 0.20) and its training error goes up (about 0.09 to 0.18). The gap between them shrinks, which means the tree overfits less and generalizes better with more data.

- (e) **Tweak the classifier** (15 points + 5 extra points if you win the contest!)—`<username>.csv`
Now let us experiment (and push yourself) a bit more by holding a Titanic competition. For this contest, we have provided a held-out test data set, `titanic_test.csv`, that is missing all information about passenger survival. Staff has the true survival information for these passengers, and your goal is to minimize the prediction error on this data set.

You should submit your predictions in a file called `<username>.csv` in your submission. This file should include your predictions on the test data set, which can be written with `write_predictions()`. **Include in your writeup a short description of what you did, including any rationale.** We will compare your predictions to ground truth and award extra credit based on how well it performs!

Ground rules:

- *Your predictions must be auto-generated from your code and use only the provided data sets.* In particular, no looking up answers on the internet (which defeats the purpose of this exercise).
- *You must use a Decision Tree classifier.* Now that you are familiar with the basics of decision trees and evaluation techniques, try your hand at improving your model. Feel free to experiment with feature combinations or decision tree parameters. You must try at least one variation on the exercises in this assignment.

Note: It is okay to try something simple. It is also okay if it turns out that your new method performs worse than the original one—just explain and document your results.

As my variation, I changed the splitting criterion from entropy to Gini impurity while keeping `max_depth=3`. I chose Gini because it is the other impurity measure covered in class, and I wanted to see whether measuring node purity differently would change the tree's splits and improve accuracy. Gini gave an average test error of 0.198 compared with 0.204 for entropy, so I trained this final model on all of the training data and used it to generate predictions for `titanic_test.csv`.

What to turn in

☐ PostCommit

- ☐ Review all modified files for quality, and add your name to README.md and all source files.
- ☐ Submit your code to your cloned GitHub repo for this assignment using the following commands.

For more details, see [GitHub's documentation](#).

```
git add .  
git commit -m "adding files for homework"  
git push
```

☐ Gradescope

- ☐ Finish the survey questions below..
- ☐ Upload your answers to (1) the written questions, (2) survey questions, and (3) any plots/figures to [Gradescope](#).
- ☐ Tag the pages to match each question in Gradescope.

Survey

1. What is your GitHub username?

aisaiahmaguadog

2. Number of hours spent working on this assignment:

It took me 6.5 hours to finish the assignment.

3. (Optional) Feel free to let me know what you liked/disliked about this homework, what you learned, etc:

I liked getting to see everything visually. The depth plot made overfitting way easier to understand than just looking at numbers. Making the tree with GraphViz was also pretty cool.